

Documentação do Projeto - ProGames

Tema e objetivo

O tema escolhido foi uma locadora de jogos chamada ProGames, atendendo video games e board games. O objetivo é permitir o cadastro de usuários, categorias e jogos, além do controle de empréstimos, devoluções, estoque, relatórios, importação/exportação de dados e logs.

Regras de negócio principais

- Um usuário pode possuir vários empréstimos.
- Uma categoria pode classificar vários jogos.
- Um empréstimo pode possuir vários jogos, e um jogo pode aparecer em vários empréstimos.
- A relação N:N entre empréstimos e jogos é resolvida pela tabela `itens\_emprestimo`.
- Ao criar um empréstimo, o estoque dos jogos é reduzido automaticamente.
- Ao devolver um empréstimo, o estoque é restaurado automaticamente.
- O valor do aluguel é calculado por diária e recebe desconto de R\$ 15,00 a partir de 3 diárias.
- Se a devolução ocorrer após a data prevista, a multa é calculada em R\$ 2,00 por dia e por item.
- Não é permitido emprestar jogo sem estoque suficiente.
- As rotas privadas exigem token JWT.
- Rotas administrativas exigem `tipo\_usuario = admin`.
- Usuários comuns acessam catálogo, reservas, histórico e perfil.

Arquitetura MVC + Service Layer

O projeto foi separado nas camadas solicitadas pelo enunciado:

- `routes`: mapeia URLs e verbos HTTP.
- `middlewares`: autentica, valida, registra logs e trata erros.
- `controllers`: recebem a requisição e retornam JSON padronizado.
- `services`: concentram regras de negócio.
- `daos`: executam consultas SQL preparadas e acesso ao MongoDB.
- `models`: representam entidades do domínio.
- `config`: centraliza configurações de MySQL, MongoDB e ambiente.

Fluxo:

Request -> Router -> Middleware -> Controller -> Service -> DAO -> Banco -> Response

Interfaces

Como JavaScript não possui interfaces nativas iguais a Java/C#, foram criadas classes didáticas que funcionam como contratos:

- `IDAO`
- `IService`
- `IController`

As classes concretas estendem esses contratos e implementam os métodos esperados, como `listar`, `buscarPorId`, `criar`, `atualizar` e `remover`.

Services

Principais services:

- `AuthService`: login, logout, bcrypt e JWT.
- `UsuarioService`: CRUD de usuários e hash de senha.
- `CategoriaService`: CRUD de categorias.
- `JogoService`: CRUD de jogos e validação de categoria.
- `EmprestimoService`: criação, devolução, multa e regras de estoque.
- `DashboardService`: dados agregados para gráficos.
- `ExportacaoService`: exportação/importação JSON.
- `LogService`: logs no MongoDB e exportação XML.
- `RelatorioService`: PDFs com PDFKit.

Controle de acesso

O projeto possui ``auth_middleware`` para validar o token JWT e ``role_middleware`` para conferir permissões por tipo de usuário. O admin acessa dashboard, cadastros, relatórios, importação/exportação e logs. O usuário comum acessa catálogo, minhas reservas, histórico e perfil.

Banco MySQL

Tabelas:

- ``usuarios``
- ``categorias``
- ``jogos``
- ``emprestimos``
- ``itens_emprestimo``

Relacionamentos:

- ``usuarios`` 1:N ``emprestimos``
- ``categorias`` 1:N ``jogos``
- ``emprestimos`` N:N ``jogos`` por ``itens_emprestimo``

O script ``database/banco.sql`` cria o banco, tabelas, chaves primárias, chaves estrangeiras, índices e inserts de teste.

MongoDB

Os logs ficam na coleção ``logs``, com campos:

- ``usuario``
- ``endpoint``
- ``metodo``
- ``timestamp``
- ``status_code``
- ``acao``
- ``ip``
- ``detalhes``

Eventos registrados:

- login/logout
- acesso a rotas
- inclusão de registros
- alteração de dados
- exclusão de dados
- erros e exceções

Quando o MongoDB não estiver iniciado, o projeto grava uma cópia temporária em ``backend/logs/logs_fallback.jsonl``. Para atender ao requisito principal, basta iniciar o MongoDB e a aplicação passa a registrar na coleção ``logs``.

Exportação XML

A rota ``GET /api/logs/exportar/xml`` consulta os logs no MongoDB e usa ``xml2js`` para gerar XML com estrutura organizada. Existem filtros opcionais por usuário e data.

Relatórios PDF

O projeto usa PDFKit para gerar:

- Relatório de jogos
- Relatório de empréstimos

Os PDFs possuem título, data/hora, usuário que gerou, tabela e totais.

Gráfico

O dashboard usa Chart.js. Os dados vêm do MySQL pela rota ``GET /api/dashboard/resumo``, mostrando jogos

mais alugados e metricas administrativas.ð

Importacao e exportacao JSONð

Rotas:ð

- `GET /api/exportacao/:entidade`ð
- `POST /api/importacao/:entidade`ð

As entidades permitidas sao:ð

- usuariosð
- categoriasð
- jogosð
- emprestimosð

A importacao recebe arquivo pelo frontend usando `input type="file"`.ð

Como executarð

1. Rodar `npm install`.ð
2. Copiar `.env.example` para `.env`.ð
3. Executar `database/banco.sql` no MySQL.ð
4. Iniciar o MongoDB, se ele estiver instalado.
5. Rodar `npm start`.ð
6. Acessar `http://localhost:3000`.ð

Usuarios de teste:

admin@locadora.com / 123456

usuario@locadora.com / 123456

Endpointsð

Principais rotas:ð

- `POST /api/auth/login`ð
- `POST /api/auth/logout`ð
- `GET /api/usuarios`ð
- `POST /api/usuarios`ð
- `PUT /api/usuarios/:id`ð
- `DELETE /api/usuarios/:id`ð
- `GET /api/categorias`ð
- `POST /api/categorias`ð
- `PUT /api/categorias/:id`ð
- `DELETE /api/categorias/:id`ð
- `GET /api/jogos`ð
- `POST /api/jogos`ð
- `PUT /api/jogos/:id`ð
- `DELETE /api/jogos/:id`ð
- `GET /api/emprestimos`ð
- `POST /api/emprestimos`ð
- `PUT /api/emprestimos/:id`ð
- `POST /api/emprestimos/:id/devolver`ð
- `DELETE /api/emprestimos/:id`ð
- `POST /api/reservas`ð
- `GET /api/minhas-reservas`ð
- `DELETE /api/minhas-reservas/:id`ð
- `GET /api/dashboard/resumo`ð
- `GET /api/exportacao/:entidade`ð
- `POST /api/importacao/:entidade`ð
- `GET /api/logs`ð
- `GET /api/logs/exportar/xml`ð
- `GET /api/relatorios/jogos/pdf`ð
- `GET /api/relatorios/emprestimos/pdf`ð